Внимание и Transformer

Как одна позиция получает информацию от остальных

Зачем обращаться к другим позициям

Слайд 1. Внимание и Transformer

Механизм attention позволяет построить новое представление позиции, смешав информацию из нескольких доступных позиций. Степень их участия определяется весами. Одни значения внесут большой вклад, другие — маленький; при определённых ограничениях некоторые вообще не будут использоваться.

Слайд 2. Нужная деталь бывает далеко

Представим длинное обращение. В начале студент указал ошибку 403, потом перечислил несколько попыток входа и в конце спросил, что делать. Для ответа может понадобиться именно номер из начала сообщения. В рекуррентной сети сведения о нём проходят через цепочку скрытых состояний. Хотелось бы иметь более прямой способ обратиться к нужной части текста.

Возьмём знакомое слово «ключ». В сочетаниях «ключ от комнаты» и «ключ доступа к сервису» его окружение помогает различить значения. Attention даёт модели способ использовать такое окружение при вычислении представления слова. Какие связи окажутся полезны, модель выясняет при обучении. Сегодня мы разберём само вычисление на небольших числах, чтобы каждый шаг можно было проследить вручную.

Здесь есть связь с поиском из третьей лекции. Там запрос сравнивался с векторами документов, после чего мы получали упорядоченный список. В attention запрос тоже сопоставляется с несколькими векторами, но итогом обычно становится взвешенная сумма значений. Мы не просто возвращаем первую подходящую позицию: можем одновременно использовать несколько.

В self-attention запросы и доступная информация происходят из одной последовательности. Каждая позиция обновляется с учётом других разрешённых позиций этой последовательности. В cross-attention запросы приходят из одного источника, а ключи и значения — из другого. Например, при переводе текущая позиция выходного текста может обращаться к представлениям исходного предложения.

Прежде чем писать общую формулу, пройдём путь одного запроса. Выясним, с чем его сравнивают, как получают веса и что именно затем суммируют. После этого

будет проще увидеть ту же операцию в матричном виде и понять, зачем в неё добавляют маски.

Запросы, ключи и значения

Слайд 3. Из X получаем Q, K и V

Пусть вход представлен матрицей X. В каждой строке находится вектор одной позиции: если токенов N, а координат d, размер X равен N×d. С помощью трёх обучаемых преобразований получим три набора векторов:

```
Q = XW_Q
K = XW_K
V = XW_V
```

Q называют запросами, K — ключами, V — значениями. Запрос определяет, по каким признакам текущая позиция будет искать информацию. Ключи участвуют в сравнении с ним. Значения содержат то, что мы затем смешаем в новое представление. Все три набора могут происходить из одного X, но выполняют разные роли.

Почему ключи и значения разделены? Удобно различать два действия: решить, к чему обратиться, и получить содержимое выбранного источника. Например, в библиотечном поиске заголовок и описание помогают выбрать книгу, но найденная информация берётся из её текста. В attention это разделение выражено двумя проекциями. Оно не означает, что координаты можно буквально подписать «тема», «автор» или «смысл»: значения координат определяются обучением.

Слайд 4. Размеры подсказывают порядок

Запрос и ключ должны иметь одинаковую размерность d_k, чтобы можно было вычислить их скалярное произведение. Размерность значения d_v может быть другой. Если N запросов сравниваются с M ключами, таблица оценок имеет размер N×M. У каждого запроса будет своя строка: по одной оценке для каждого доступного ключа.

Значения образуют матрицу $M×d_v$. После вычисления весов мы умножим таблицу N×M на значения $M×d_v$ и получим $N×d_v$. Так каждая строка выхода станет новым вектором соответствующего запроса. Например, два запроса при трёх ключах дают таблицу оценок 2×3. Умножение на три строки значений возвращает два выходных вектора.

В self-attention обычно N=M, потому что речь об одной последовательности. В cross-attention числа могут различаться: например, длина перевода не обязана совпадать с длиной исходного предложения. Размерности показывают, что именно сопоставляется и какие строки участвуют в сумме.

Вычислим сначала только первую строку. Если этот шаг понятен, остальные запросы обрабатываются таким же способом; матричная запись просто выполняет все сравнения и суммы сразу.

От оценок к новому представлению

Слайд 5. Два запроса, два ключа

В scaled dot-product attention сначала вычисляют скалярные произведения запросов и ключей, затем делят оценки на $\sqrt{d_k}$. К каждой строке применяют softmax и получают веса. Последнее действие — умножение этих весов на значения:

```
S = QKᵀ / √dₖ
A = softmax(S)      по строкам, вдоль ключей
O = AV
```

Масштабирование помогает удерживать разброс оценок при изменении размерности. Если оценки сильно отличаются, softmax может дать почти весь вес одному элементу. Деление на $\sqrt{d_k}$ контролирует их характерный масштаб. Это не нормировка каждого вектора по его длине, как в косинусе: в формуле выполняется другая операция.

Возьмём два запроса и два ключа: Q=K — единичная матрица 2×2. Первый вектор равен (1; 0), второй — (0; 1). Значения зададим отдельно: V_1=(10; 0), V_2=(0; 20). Здесь d_k=2.

Скалярное произведение первого запроса с первым ключом равно 1, со вторым — 0. После деления на $\sqrt{2}$ строка оценок становится (0,7071; 0), если округлить до четырёх знаков.

Слайд 6. Из оценок получаются веса

Теперь softmax. Экспонента от 0,7071 приблизительно равна 2,028, от нуля — 1. Делим каждое число на их сумму и получаем веса около (0,6698; 0,3302). Они складываются в 1. Нулевая исходная оценка при этом получила положительный вес: exp(0)=1, поэтому её вклад не исчез.

Слайд 7. Выход — взвешенная сумма

Новое значение для первой позиции — 0,6698×(10; 0)+0,3302×(0; 20). При расчёте с неокруглёнными весами результат приблизительно равен (6,6976; 6,6048). Первая часть суммы дала вклад в первую координату, вторая — во вторую. У второго значения вес меньше, но его ненулевая координата вдвое больше; поэтому вклады в выход оказались близки по величине.

Проследите весь расчёт: q_1 сравнивается с k_1 и k_2, оценки превращаются в два веса, каждый вес умножается на соответствующее V, затем складываются координаты. Если пропустить последний шаг и смотреть только на веса, получится неполное представление о результате. Веса определяют смешивание, но содержание зависит от самих значений.

Слайд 8. Расчёт для всех запросов

Для второго запроса в нашем примере веса меняются местами: (0,3302; 0,6698). Его выход приблизительно равен (3,3024; 13,3952). Полная матрица A имеет две строки; сумма элементов каждой строки равна 1. Одна строка относится к первому запросу, другая — ко второму.

Симметрия здесь получилась из выбранных чисел. В общем случае вес связи из i в j не обязан совпадать с весом из j в i: запросы различаются, а нормировка выполняется отдельно в каждой строке. Это стоит помнить при чтении тепловой карты внимания — она изображает направленные отношения между запросами и ключами.

В коде softmax удобно вычислять устойчивым способом: вычесть максимум строки, взять экспоненты, разделить на сумму. Общее смещение оценок не меняет результат. Например, softmax для (2; 1; 0) и (1002; 1001; 1000) должен совпасть, хотя прямое вычисление экспонент во втором случае создаёт численные трудности.

Какие позиции разрешено использовать

Слайд 9. Будущее закрыто до softmax

До сих пор каждый запрос мог обращаться ко всем ключам. Для анализа готового предложения это удобно: у слова есть левое и правое окружение. Но при авторегрессионном предсказании будущие токены ещё неизвестны. Если показать их модели во время обучения, она сможет использовать сведения, которых не будет при генерации.

Причинная, или causal, маска разрешает позиции i обращаться к ключам $j \le i$ и закрывает $j > i$. Диагональ остаётся доступной: представление текущего входного токена используется для предсказания следующего. В таблице это нижний треугольник вместе с диагональю.

Запрет применяется к оценкам до softmax. На месте недоступного ключа ставят $-\infty$. Его экспонента равна нулю, поэтому после нормировки получается нулевой вес. Просто заменить оценку на 0 недостаточно — мы только что видели, что такая оценка даёт положительный вклад.

С причинной маской первой позиции разрешён только первый ключ. В нашем примере строка оценок становится (0,7071; −∞), а веса — (1; 0). Первый выход

теперь равен V_1=(10; 0). Вторая позиция всё ещё может обращаться к обоим ключам и сохраняет свой прежний результат.

Слайд 10. Что изменится при замене V2?

Проверим понимание. Пусть мы изменили V_2, оставив Q, K, V_1 и маску прежними. Что произойдёт с первым выходом?

Слайд 11. Ранний выход не изменился

При корректной маске выход первой позиции не меняется: изменённое значение по-прежнему умножается на нулевой вес. Это удобно проверить в лабораторной — такое изменение выявляет ошибку, если будущее оказалось случайно доступным.

Слайд 12. Padding исключаем из ключей

Другой вид маски нужен для padding. Чтобы обработать несколько текстов одним пакетом, короткие последовательности часто дополняют служебными позициями до общей длины. Эти позиции не содержат сообщения пользователя. Маска ключей исключает их из источников информации.

Причины запрета различаются: один ключ находится в будущем, другой является заполнителем. Ограничения можно объединить. В используемой нами функции True будет означать доступный ключ. Перед применением готовой библиотеки нужно проверить её соглашение: в разных интерфейсах булевы маски трактуются по-разному.

Слайд 13. Когда все ключи закрыты

Что произойдёт, если в строке закрыть все ключи? Экспоненты будут равны нулю, и нормировать их не получится: знаменатель тоже нулевой. В учебной реализации такой вход лучше явно отклонить. Это помогает отличить ошибку построения маски от содержательного поведения модели.

Откуда модель узнаёт порядок

Слайд 14. Перестановка входа и выхода

Для следующего опыта временно уберём маски: все позиции смогут обращаться ко всем ключам. В формуле QK^T нет номера позиции. Если одинаково переставить входные строки и не добавить позиционную информацию, такой self-attention переставит соответствующие выходные строки. Сами вычисления будут следовать за векторами. Это называют перестановочной эквивариантностью. Если П обозначает перестановку строк, то без позиций и ограничений

Attention(ПX)=ПAttention(X). Неизменная причинная маска, напротив, уже различает доступ по положению и не подходит для этой проверки.

Вернёмся к фразам «Преподаватель консультирует студента» и «Студент консультирует преподавателя». Для определения ролей участников полезны и словоформы, и порядок. Механизму нужен способ различать, где находится элемент последовательности, помимо содержания его вектора.

Слайд 15. Позиции остаются на местах

Один из способов — добавить к представлению токена позиционный вектор: H=X+P. У одной и той же словоформы в разных местах вход теперь будет различаться. В исходной статье Transformer использовались синусоидальные позиционные представления; существуют и обучаемые, и другие способы задавать положение. Их общая задача — сделать порядок доступным вычислению.

Здесь важно аккуратно поставить опыт с перестановкой. Без позиций переставляем входные строки, получаем выход, возвращаем строки в исходный порядок и сравниваем с прежним результатом. С позиционными векторами отдельно решаем, что именно переставили: содержимое токенов или токены вместе с прежними позиционными признаками. Это разные входы и разные проверки.

В лабораторной мы сначала увидим совпадение выходов после обратной перестановки без позиций. Затем добавим позиционные векторы и изменим порядок токенов, оставив позиционный вектор на каждом месте прежним. Теперь результат может измениться. Так смысл позиции проявится в числах, а не только в подписи под архитектурой.

Зачем несколько голов

Слайд 16. У каждой головы свои проекции

Одна голова attention использует свои проекции запросов, ключей и значений. Если сделать несколько голов, каждая сможет выполнять сравнение в своём представлении. У всех один исходный текст, но обучаемые матрицы различаются.

Представьте, что для нового представления слова одновременно полезны несколько связей с контекстом. Один общий набор весов может смешать их неудобным образом. Несколько голов дают несколько отдельных вычислений, результаты которых затем объединяются. При этом мы не назначаем голове заранее роль вроде «ищет существительные» или «проверяет отрицание». Такие свойства, если они появляются, нужно исследовать на обученной модели.

Слайд 17. Соединяем результаты голов

Для каждой головы вычисляются свои Q, K, V и свой выход. Затем выходы конкатенируют по координатам и применяют выходную проекцию W_O. Если у головы N строк и d_v координат, у h голов после конкатенации будет $N \times (h \cdot d_v)$. Проекция переводит это представление в нужную ширину модели. Например, у трёх голов с двумя координатами в каждой после конкатенации будет шесть координат для одной позиции.

В одной голове вход проходит через один набор проекций, а в нескольких головах — через несколько параллельных наборов. Различается не число копий исходного текста в задаче, а число обучаемых преобразований одного входа. Слияние результатов — полноценная часть механизма: оно позволяет использовать информацию, полученную разными головами.

Количество голов нельзя само по себе считать оценкой качества. Для нас сейчас важнее суметь восстановить размеры промежуточных массивов и показать, где они разделяются и снова соединяются. Это пригодится при чтении конфигурации настоящего энкодера на следующем занятии.

Что ещё находится в блоке Transformer

Слайд 18. Внутри блока Transformer

Attention — часть блока, а не весь Transformer. После обмена информацией между позициями модель также преобразует признаки каждой позиции. Для этого используется feed-forward сеть, обычно с нелинейностью между линейными слоями.

В типичном варианте такая сеть применяется к каждой позиции отдельно и с общими параметрами. Если attention отвечает за взаимодействие позиций, feed-forward меняет представление внутри строки. На схеме эти действия удобно разделять: одно смешивает информацию между токенами, другое преобразует признаки каждого токена.

Остаточная связь добавляет вход подблока к его результату. Благодаря этому у представления остаётся прямой путь через блок; преобразование не обязано заново воспроизводить всё исходное содержимое. Сумма требует одинаковых размерностей, поэтому положение выходных проекций имеет значение.

Нормировка помогает управлять масштабом промежуточных значений. В рассматриваемом блоке Norm означает LayerNorm. Она работает с координатами внутри вектора каждой позиции: вычитает их среднее и делит на корень из дисперсии с добавленным небольшим ε. Затем применяются обучаемые масштабы и смещения. Другие позиции и другие тексты пакета в это вычисление среднего не входят. В отличие от изученной L2-нормировки, здесь не требуется сделать евклидову длину вектора равной единице.

Например, у вектора (2; 4) среднее равно 3, а дисперсия — 1. После центрирования и масштабирования получатся значения, близкие к (−1; 1), если пока не учитывать обучаемые параметры и малое ε. Длина такого вектора близка к $\sqrt{2}$, а не к единице. Этот пример помогает различать LayerNorm и L2-нормировку эмбеддинга.

Положение нормировки различается между вариантами архитектуры. Разберём pre-norm запись, где она стоит перед соответствующим подблоком. Attention в этой записи включает объединение голов и выходную проекцию: её результат имеет ту же ширину, что и X, поэтому остаточная сумма определена.

```
H = X + Attention(Norm(X))
Y = H + FFN(Norm(H))
```

Здесь видны две остаточные суммы. Сначала attention получает нормированный вход, а результат прибавляется к X. Затем нормируется H, применяется FFN и его результат прибавляется к H. Каждый обходной путь сохраняет вход своего подблока и передаёт его к соответствующей сумме. Если прибавить другой промежуточный вектор, вычисление изменится.

В полном Transformer такие блоки повторяются. После каждого слоя представления позиций обновляются, поэтому внимание следующего слоя работает уже с другим входом. Это позволяет последовательно строить более сложные преобразования. Одной карты весов первого слоя недостаточно, чтобы восстановить весь путь к окончательному ответу модели.

Энкодер, декодер и связь между ними

Слайд 19. Разная доступность контекста

Энкодер обрабатывает доступную последовательность целиком. Для представления слова он может использовать левое и правое окружение, кроме заполнителей PAD, закрытых маской. Такой вариант подходит для получения признаков текста, классификации и выделения сущностей. На следующей лекции мы будем использовать предобученный BERT-энкодер.

Авторегрессионный декодер работает с причинным ограничением. Представление позиции строится по доступному префиксу и используется для предсказания следующего токена. Именно поэтому при обучении нужна та же логика доступности, что и при генерации. Большой decoder-only Transformer остаётся языковой моделью следующего токена, хотя его поведение на инструкциях может быть сложным.

Слайд 20. Cross-attention связывает два ряда

В encoder-decoder архитектуре исходная последовательность сначала проходит через энкодер. Декодер строит выход, используя собственный префикс и cross-

attention к выходам энкодера. В таком cross-attention запросы происходят из состояния декодера, а ключи и значения — из представлений исходного текста.

Посмотрим на перевод. Уже созданные слова перевода задают текущее состояние декодера. Для следующего шага полезно обратиться к определённым частям исходного предложения. Cross-attention связывает два ряда представлений. Это отличается от self-attention декодера, где он обращается к собственному доступному префиксу.

Названия семейств помогают ориентироваться, но выбирать модель нужно с учётом того, на какой задаче и каким способом она обучалась. Само наличие блока attention ещё не говорит, какие входы ожидаются и как получить нужный выход. Мы увидим это на практике, когда загрузим конкретный checkpoint и его токенизатор.

Размер матрицы и смысл весов

Слайд 21. Квадратичный рост

Если N запросов сравниваются с N ключами, таблица оценок содержит N^2 элементов. При увеличении длины в четыре раза число пар увеличивается в шестнадцать. Это видно геометрически: сторона квадратной таблицы выросла в четыре раза, а площадь — в шестнадцать.

Посчитаем память одной такой матрицы в float32, по четыре байта на элемент. Для длины 1024 это $1024^2 \times 4$ байта, то есть 4 MiB. Для длины 4096 — 64 MiB. Эти числа относятся к одной матрице, без остальных промежуточных массивов и градиентов. Некоторые эффективные реализации избегают хранения полной матрицы целиком; из размера математической таблицы нельзя напрямую получить весь расход памяти программы.

Слайд 22. Разные веса — тот же выход

Есть и другой важный вопрос: можно ли считать большой вес объяснением ответа? Вес действительно показывает вклад строки V в конкретную сумму. Но итог зависит и от значений, и от других голов, проекций, остаточных связей и последующих слоёв.

Рассмотрим простой случай: $V_1=V_2=(1; 2)$. Возьмём веса (0,9; 0,1), затем (0,2; 0,8). В обоих случаях взвешенная сумма равна (1; 2). Карты весов разные, а выход одинаков. По одной картинке весов нельзя заключить, что изменение указанной позиции обязательно изменило бы конечный результат.

Для анализа полезны контролируемые изменения входа и наблюдение за выходом. Именно так мы проверяли маску: меняли только значение закрытой позиции и смотрели на первый результат. Такой опыт отвечает на конкретный вопрос о

вычислении. Он содержательнее общего утверждения «модель обратила внимание на важное слово» без указания, что было измерено.

Что будем делать в лабораторной

Слайд 23. Что проверить в реализации

В лабораторной мы реализуем небольшой attention на NumPy. Сначала проверим softmax на простых числах, затем воспроизведём пример с двумя запросами, ключами и значениями. После этого добавим маски и посмотрим, какие изменения входа перестают влиять на результат.

Три проверки особенно полезны. В каждой допустимой строке веса должны складываться в единицу. Недоступный ключ должен получать нулевой вес. Изменение будущего значения при причинной маске не должно менять ранний выход. Все три свойства можно связать с теми шагами, которые мы только что выполнили вручную.

Далее сравним перестановки без позиций и с позиционной информацией. В конце вернёмся к двум одинаковым значениям и разным картам весов. В результате у нас будет не просто функция с правильным именем, а понимание того, что именно происходит внутри неё и как заметить ошибку.

Слайд 24. От ручных матриц к модели

На следующем занятии вместо маленьких матриц мы возьмём готовый энкодер. Размеры станут больше, а параметры будут получены предобучением, но запросы, ключи, значения и маски сохранят тот смысл, который мы разобрали сегодня.

Для дальнейшего чтения

– Vaswani et al. Attention Is All You Need — архитектура Transformer и scaled dot-product attention.

– Lena Voita. Seq2seq and Attention — последовательные визуальные разборы внимания.

– Jain, Wallace. Attention Is Not Explanation — ограничения интерпретации весов внимания.

– Dao et al. FlashAttention — пример организации вычисления, уменьшающей обращения к памяти и хранение промежуточных результатов.